Python高级与正则表达式

人生苦短，我学Python!

黑马程序员
www.itheima.com
传智教育旗下
高端IT教育品牌

目录

Contents

- 1、迭代器
- 2、yield生成器
- 3、正则表达式

01 Python迭代器

黑马程序员
www.itheima.com

1、什么是迭代器

迭代器（Iterator）是 Python 中的一种对象，用于在数据集合中逐个访问元素，而不需要暴露数据集合的底层实现。它提供了一种遍历集合元素的标准方式，适用于任何支持迭代的数据结构，如列表、元组等，range()就是一个迭代器

迭代器是一个实现了 __iter__() 和 __next__() 方法的对象，使得可以逐步遍历它的元素。

特点：

手动管理：需要显式地实现 __iter__() 和 __next__() 方法。

状态管理：迭代器需要自己管理迭代的状态，包括当前位置和结束条件。

内存使用：内存使用取决于迭代器的实现，通常是惰性计算（即按需生成数据）。

黑马程序员
www.itheima.com

2、迭代器底层实现

```
class MyIterator:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self
    def __next__(self):
        if self.current > self.end:
            raise StopIteration
        value = self.current
        self.current += 1
        return value

# 使用迭代器
iterator = MyIterator(1, 5)
for num in iterator:
    print(num)
```

总结

1. 定义：迭代器是一个对象，提供了一个访问集合元素的方式，不暴露集合的底层实现。
2. 协议：需要实现 __iter__() 和 __next__() 方法
3. 使用：可以通过 for 循环或 next() 函数进行遍历。
4. 优势：支持惰性计算和高效的内存使用，适合处理大型数据集合或流数据。

02 Python生成器

黑马程序员
www.itheima.com

1、什么是生成器

根据程序员制定的规则循环生成数据，当条件不成立时则生成数据结束。数据不是一次性全部生成出来，而是使用一个，再生成一个，可以**节约大量的内存**。

创建生成器的方式：① 生成器推导式 ② yield 关键字

黑马程序员
www.itheima.com

2、生成器推导式

- # 创建生成器

```
# 创建生成器
# 注意1：括号（）代表 这是一个生成器，不是元组
# 注意2：括号（）里面写的是数据的生成规则，返回一个对象，
                                    # 对象内不是存的数据，而是产生数据的规则
my_generator = (i * 2 for i in range(5))    # 根据注意2
print(my_generator)

# next获取生成器下一个值
# value = next(my_generator)
# print(value)

# 遍历生成器
for value in my_generator:
    print(value)
```

- 生成器相关函数：

next 函数获取生成器中的下一个值

for 循环遍历生成器中的每一个值

黑马程序员
www.itheima.com

3、yield生成器

只要在def函数里面看到有 yield 关键字那么就是生成器

```
def mygenerater(n):
    for i in range(n):
        print('开始生成...')
        yield i
        print('完成一次...')
```

```
if __name__ == '__main__':
    g = mygenerater(2)
    # 获取生成器中下一个值
    # result = next(g)
    # print(result)

    # for遍历生成器，for 循环内部自动处理了停止迭代异常，使用起来更加方便
    for i in g:
        print(i)
```

黑马程序员
www.itheima.com

4、生成器的应用场景 - 数据迭代器dataloader的封装

```
import math
def dataset_loader(bath_size):
    # 1 读歌词
    with open('./jaychou_lyrics.txt', 'r') as file :
        lines = file.readlines()

    # 2 统计共有多少条歌词
    lyrics_number = len(lines)

    # 3 计算共有多少个批次 math.ceil向上取整 math.floor向下取整
    batch_number = math.ceil(lyrics_number/bath_size)

    # 4 遍历每一个 batch
    for idx in range(batch_number):
        yield lines[idx*bath_size : idx*bath_size+bath_size]
```

- 很多模型都是一个批次一个批次的给模型喂数据，来训练模型的。
- 构建数据生成器每8个条数据（8个样本）8个数据的给模型喂数据

```
if __name__ == '__main__':
  dataloader = dataset_loader(8)
  for data in dataloader:
    print(data)

  print('创建生成器(数据加载器) 为AI专业课做准备 End')
```

总结

1 生成器的概念？ 如何使用?

- 根据一定规则生成数据的一种机制，每次调用生成器只生成一个值，可以节省大量内存。

- next(generater) next 函数获取生成器中的下一个值
- for 循环遍历生成器中的每一个值

2 生成器的创建有两种方式?

- 生成器推导式
- yield 关键字

3 yield关键字的作用（特性）

- 1 将yield后面的值返回
- 2 在yield这个地方卡着（阻塞）

总结

生成器和迭代器区别？

1. 实现方式

迭代器：需要实现 __iter__() 和 __next__() 方法，手动管理迭代状态。

生成器：通过 yield 关键字简化实现，自动管理迭代状态。

2. 代码复杂度

迭代器：通常需要更多的代码来管理状态和迭代逻辑。

生成器：代码更简洁，更容易理解和维护。

3. 性能与内存

迭代器：性能和内存使用取决于实现，通常也是惰性计算。

生成器：由于使用了 yield，内存使用和性能优化自动管理，适合处理大数据或流数。

4. 使用场景

迭代器：适合需要对迭代过程进行高度控制，或者需要自定义复杂的迭代逻辑时使用

生成器：适合需要简洁地生成序列数据，尤其是在处理大数据或需要按需生成数据时，能够节省内存和提高性能

02 Property属性

Python高级语法

1、property属性的介绍

- 负责把一个方法当做属性进行使用，这样做可以简化代码使用。

- 定义property属性有两种方式：① 装饰器方式 ② 类属性方式

Python高级语法

黑马程序员 www.itheima.com

2、装饰器方式

```
class Person(object):
    def __init__(self):
        self.__age = 0

    # 获取属性
    @property
    def age(self):
        return self.__age

    # 修改属性
    @age.setter
    def age(self, new_age):
        self.__age = new_age
```

```
if __name__ == '__main__':
    p1 = Person()
    print(p1.age)

    p1.age = 100
    print(p1.age)
```

 # 装饰器方式:

 # @property 修饰获取值的方法

 # @方法名.setter 修饰设置值的方法

Python高级语法

黑马程序员 www.itheima.com

3、类属性方式

```
class Person(object):
    def __init__(self):
        self.__age = 0


    def get_age(self):
        """当获取age属性的时候会执行该方法"""
        return self.__age

    def set_age(self, new_age):
        """当设置age属性的时候会执行该方法"""
        self.__age = new_age

    # 类属性方式的property属性
    age = property(get_age, set_age)
```

```
if __name__ == '__main__':
    p1 = Person()
    print(p1.age)


    p1.age = 100
    print(p1.age)
```

 # 类属性方式:

 # 类属性 = property(获取值方法, 设置值方法)

总结

1 property属性的作用？

- 把一个方法当做属性进行使用，这样做可以简化代码使用

2 定义property属性的两种方式，各自的语法？

装饰器方式:

① @property 修饰获取值的方法

② @方法名.setter 修饰设置值的方法

类属性方式:

① property属性 = property(获取值方法, 设置值方法)

04 正则表达式

正则表达式概述

黑马程序员 www.itheima.com

1、为什么要学习正则表达式

在实际开发过程中经常会有查找符合某些规则的字符串

比如：邮箱、图片地址、手机号码等。想匹配或者查找符合某些规则的字符串就可以使用正则表达式了。

邮箱：abc@itcast.cn

手机号：13888888888

正则表达式概述

2、什么是正则表达式

正则表达式(regular expression)描述了一种字符串匹配的模式，

1、比如：检索一个串是否含有某种子串（检索）

2、比如：匹配的子串做替换（替换）

3、比如：从一个串中取出符合某个条件的子串（提取）

模式：一种特定的字符串模式，这个模式是通过一些特殊的符号组成的。

正则表达式并不是Python所特有的，在Java、PHP、Go以及JavaScript等语言中都是支持正则表达式的。

正则表达式概述

3、正则表达式的功能

① 数据验证（表单验证、如手机、邮箱、IP地址）

② 数据检索（数据检索、数据抓取） => 爬虫功能

③ 数据隐藏（135****6235 王先生）

④ 数据过滤（论坛敏感关键词过滤）

…

05 re模块的介绍

正则表达式概述

黑马程序员
www.itheima.com

1、什么是re模块

在Python中需要通过正则表达式对字符串进行匹配时，可使用re模块

正则表达式概述

2、re模块使用三步走

```
# 第一步：导入re模块
import re
# 第二步：使用match方法进行匹配操作
result = re.match(pattern正则表达式, string要匹配的字符串, flags=0)
#flags：可选，表示匹配模式，比如忽略大小写，多行模式等
# 第三步：如果数据匹配成功，使用group方法来提取数据
result.group()
```

正则表达式概述

黑马程序员 www.itheima.com

3、举个栗子 -1

```
def dm01_match匹配字符():

    “”“匹配字符：从大字符串中，按照规则，匹配符合条件的子串”“”
    # 1 导入re模块
    import re

    # 2 使用match方法进行匹配操作
    # 2-1 在大的字符串中，按照规则：“任意1个字符”+“it”+“任意1个字符”，提取符合要求的子串
    # 注意：提取出来的子串一定要符合规则
    result = re.match(".it.", "aitcast")

    # 2-2 从左到右的匹配(不能跳，不能从中间匹配)，一个字符一个字符的匹配
    # result = re.match(".it.", "iloveitcast")

    # 3 使用group方法来提取数据
    if result:
        info = result.group()
        print(info)
    else:
        print("没有找到符合规则的子串")
```

程序运行结果：aitc

正则表达式概述

黑马程序员 www.itheima.com

3、举个栗子 -2

```
def dm02_search扫描字符串():
    ''' # 扫描字符返回第一个成功的匹配 def search(pattern, string, flags=0) '''

    import re
    result = re.search("\d.*", "city:1beijing2.shanghai")  # "\d.*": 数字开头,任意多个字符字符结尾
    # result = re.search(".\d.", "cityp.1.beijing2.shanghai")
    if result:
        print(result.group())
    else:
        print('没有匹配到')
    pass
```

程序运行结果：1beijing2.shanghai

正则表达式概述

黑马程序员 www.itheima.com

3、举个栗子 -3

```
def dm03_replace替换字符串():
    import re
    sentence = "车主说:你的刹车片应该更换了啊,嘿嘿"

    # 正则表达式: 去除多余字符
    p = r"呢|吧|哈|啊|啦|嘿|嘿嘿"
    r = re.compile(pattern=p)
    mystr = r.sub('', sentence)
    print('mystr-->', mystr)

    # 正则表达: 删除除了汉字数字字母和，！？。.- 以外的字符
    # \u4e00-\u9fa5 是用来判断是不是中文的一个条件
    p = "[^，！？。\.\-\u4e00-\u9fa5_a-zA-Z0-9]"
    r = re.compile(pattern=p)
    mystr = r.sub('', sentence)
    print('mystr-->', mystr)

    # 半角变为全角 sentence.replace(",", "，") 逗号 感叹号 问号
    sentence = "你好."
    mystr = sentence.replace(".", "。")
    print('mystr-->', mystr)
```

```
mystr--> 车主说:你的刹车片应该更换了,
mystr--> 车主说你的刹车片应该更换了啊嘿嘿
mystr--> 你好。
```

05 正则表达式编写

黑马程序员
www.itheima.com

目标
TARGET

- 1、能够使用re模块匹配单个字符

正则表达式编写

1、能够使用re模块匹配单个字符

代码	功能
.	匹配任意1个字符（除了\n）
[ ]	匹配[ ]中列举的字符
[^指定字符]	匹配除了指定字符以外的所有字符
\d	匹配数字，即0-9
\D	匹配非数字，即不是数字

黑马程序员
www.itheima.com

2、能够使用re模块匹配单个字符

代码	功能
\s	匹配空白，即 空格，tab键
\S	匹配非空白
\w	匹配非特殊字符，即a-z、A-Z、0-9、_、汉字
\W	匹配特殊字符，即非字母、非数字、非汉字

正则表达式编写

黑马程序员
www.itheima.com

3、举个栗子 -1

```
# 1 . 匹配任意1个字符（除了\n）
# 匹配数据: 从左向右匹配，一个字符接着一个字符的匹配
result = re.match("itcast.", "itcast2")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

```
程序运行结果：itcast2
```

正则表达式编写

3、举个栗子 -2

```
# 2 [ ] 匹配[ ]中列举的字符
# [a-z]  [A-Z] [0-9]   [a-zA-Z0-9]
# 匹配数据
result = re.match("itcast[123abc]", "itcast376")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

程序运行结果：itcast3

正则表达式编写

黑马程序员 www.itheima.com

3、举个栗子 -3

```
# 3 \d  匹配数字,即0-9 => [0123456789] => [0-9]
# 匹配数据
result = re.match("itcast\d", "itcast5")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

```
程序运行结果：itcast5
```

黑马程序员
www.itheima.com

3、举个栗子 -4

```
# 4 \D  匹配非数字, 即不是数字
# 匹配数据
result = re.match("itcast\D", "itcast-")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

程序运行结果：itcast-

黑马程序员
www.itheima.com

3、举个栗子 -5

```
# 5 \s 匹配空白,即空格,tab键
# 匹配数据
result = re.match("itcast\s111", "itcast\t111")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

程序运行结果：itcast 111

黑马程序员
www.itheima.com

3、举个栗子 -6

```
# 6 \S 匹配非空白
# 匹配数据
result = re.match("itcast\S", "itcast\t")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

程序运行结果：没有匹配到

正则表达式编写

3、举个栗子 -7

```
# 7 \w 匹配非特殊字符，即a-z, A-Z, 0-9, _, 汉字
# 匹配数据
result = re.match("itcast\w", "itcasta")
# result = re.match("itcast\w", "itcast!")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

程序运行结果：itcasta

黑马程序员
www.itheima.com

3、举个栗子 -8

```
# 8 \W 匹配特殊字符,即非字母, 非数字, 非_, 非汉字
# 匹配数据
result = re.match("itcast\W", "itcast\t2aa")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

```
程序运行结果：itcast
```

正则表达式编写

黑马程序员
www.itheima.com

总结

代码	功能
.	匹配任意1个字符（除了\n）
[ ]	匹配[ ]中列举的字符
[^指定字符]	匹配除了指定字符以外的所有字符
\d	匹配数字，即0-9，等价于[0-9]
\D	匹配非数字，即不是数字
\s	匹配空白，即 空格，tab键
\S	匹配非空白
\w	匹配非特殊字符，即a-z、A-Z、0-9、_、汉字
\W	匹配特殊字符，即非字母、非数字、非_、非汉字

黑马程序员
www.itheima.com

目标
TARGET

- 2、能够使用re模块匹配多个字符

正则表达式编写

1、能够使用re模块匹配多个字符

代码	功能
*	匹配前一个字符出现0次或者无限次，即可有可无
+	匹配前一个字符出现1次或者无限次，即至少有1次
?	匹配前一个字符出现1次或者0次，即要么有1次，要么没有
{m}	匹配前一个字符出现m次
{m,n}	匹配前一个字符出现从m到n次

正则表达式编写

2、举个栗子 -1

```
# 1 *  匹配前一个字符出现0次或者无限次，即可有可无
result = re.match("itcast1*", "itcast111123333itcast")      # 1出现0次或者多次
# result = re.match("itcast\d*", "itcast23333itcast")        # 数字出现0次或者多次
# result = re.match("itcast\d*itcast", "itcast123333itcast")   # 数字出现0次或者多次 itcast开始 itcast结束
# result = re.match("itcast\d*itcast", "itcastitcast")   # 数字出现0次或者多次 itcast开始 itcast结束

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

程序运行结果：itcast1111

正则表达式编写

黑马程序员 www.itheima.com

2、举个栗子 -2

```
# 2 +  匹配前一个字符出现1次或者无限次，即至少有1次
# 数字必须出现1次或者多次
result = re.match("itcast\d+itcast", "itcast11333444itcast")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

程序运行结果：itcast11333444itcast

正则表达式编写

黑马程序员 www.itheima.com

2、举个栗子 -3

```
# 3？ 匹配前一个字符出现1次或者0次，即要么有1次，要么没有
# 数字出现0次或者1次
result = re.match("itcast\d?itcast", "itcastitcast")
# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

```
程序运行结果：itcastitcast
```

正则表达式编写

黑马程序员 www.itheima.com

2、举个栗子 -4

```
# 4 {m} 匹配前一个字符出现m次
result = re.match("itcast\d{2}itcast", "itcast12itcast")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

```
程序运行结果：itcast12itcast
```

正则表达式编写

黑马程序员 www.itheima.com

2、举个栗子 -5

```
# 5 {m,n}  匹配前一个字符出现从m到n次
# 注意1:{2, 5} 中括号里面,逗号之后不能加空格
result = re.match("itcast\d{2,5}itcast", "itcast111itcast")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

程序运行结果：itcast111itcast

黑马程序员
www.itheima.com

2、总结

总结

代码	功能
*	匹配前一个字符出现0次或者无限次，即可有可无
+	匹配前一个字符出现1次或者无限次，即至少有1次
?	匹配前一个字符出现1次或者0次，即要么有1次，要么没有
{m}	匹配前一个字符出现m次
{m,n}	匹配前一个字符出现从m到n次

黑马程序员
www.itheima.com

目标
TARGET

- 3、能够使用re模块匹配指定字符串开头或者结尾

正则表达式编写

黑马程序员
www.itheima.com

1、能够使用re模块匹配指定字符串开头或者结尾

代码	功能
^	匹配字符串开头
$	匹配字符串结尾

正则表达式编写

黑马程序员 www.itheima.com

2、举个栗子 -1

```
# 1-1 ^ 匹配字符串开头
# 匹配数据: 匹配1个数字开头的子串
result = re.match("^\ditcast", "2itcast")  # 1 匹配1个数字开头 + itcast
# result = re.match("^\ditcast", "22itcast")  # 2 匹配不上

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

```
程序运行结果：2itcast
```

黑马程序员
www.itheima.com

2、举个栗子 -2

```
# 1-2 以数字为开头的字符串
result = re.match("^\d.*", "22itcast")  # "^\d":以数字开头, ".*":以字符结尾
# result = re.match("^\d{1,3}it", "1itcast")  # "^\d{1,3}":以1~3个数字开头, ".*":以字符结尾

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

```
程序运行结果：22itcast
```

黑马程序员
www.itheima.com

2、举个栗子 -3

```
# 2-1 $ 匹配字符串结尾
result = re.match(".*\d$", "itcast66")  # ".*" : 0个多个字符开头, "\d$":数字结尾
# result = re.match(".*\d{5}$", "itcast666")  # ".*" : 0个多个字符开头, "\d{5}$": 以5个数字结尾

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

程序运行结果：itcast66

黑马程序员
www.itheima.com

2、举个栗子 -4

```
# 3 匹配以数字为开头以数字为结尾
result = re.match("^\d.*\d$", "11itcast22")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

```
程序运行结果：11itcast22
```

黑马程序员
www.itheima.com

2、举个栗子 -5

```
# 4 [^指定字符]  匹配除了指定字符以外的所有字符
result = re.match("^\d.*[^4]$", "11itcast@")
# result = re.match("^\d.*[^4]$", "11itcast4")

# 获取数据
if result:
    info = result.group()
    print(info)
else:
    print("没有匹配到")
```

程序运行结果：11itcast@

黑马程序员
www.itheima.com

3、总结

总结

代码	功能
^	匹配字符串开头
$	匹配字符串结尾

```
# 1 ^    匹配字符串开头    eg: ^a 子串以a开头
# 2 $    匹配字符串结尾    eg: 4$ 子串以4结尾
# 3 [^指定字符]   匹配除了指定字符以外的所有字符   eg: ^4.*[^4]$ 不是以4结尾
```

```
# 3 匹配以数字为开头以数字为结尾
result = re.match("^\d.*\d$", "11itcast22")
```

黑马程序员
www.itheima.com

目标
TARGET

- 4、能够使用re模块提取分组数据

黑马程序员
www.itheima.com

1、匹配分组

代码	功能
|	匹配左右任意一个表达式
(ab)	将括号中字符作为一个分组
\num	引用分组num匹配到的字符串

正则表达式编写

黑马程序员 www.itheima.com

2、举个栗子 -1

```
# 1 需求：在列表中["apple", "banana", "orange", "pear"]，匹配apple和pear
fruit = ["apple", "banana", "orange", "pear"]

# 获取字符串数据
for value in fruit:
    result = re.match("apple|pear", value)
    # 判断匹配是否成功
    if result:
        info = result.group()
        print("我想吃的水果:",value)
    else:
        print(f"这个不是我想吃的水果{value}")
```

```
程序运行结果：
    我想吃的水果: apple
    这个不是我想吃的水果banana
    这个不是我想吃的水果orange
    我想吃的水果: pear
```

正则表达式编写

黑马程序员 www.itheima.com

2、举个栗子 -2

```
# 2 需求：匹配出163、126、qq等邮箱
# |     匹配左右任意一个表达式
# (ab) 将括号中字符作为一个分组
# \ 转义字符

# 2-1
# result = re.match("[a-zA-Z0-9_]{4,20}@163|126|qq.com", "hello@163.com")  # 只能把"hello@163"匹配出来

# 2-2
# 不能匹配出来子串 ,因为分解成: "{4,20}@163" | "126" |  "qq.com"
# result = re.match("[a-zA-Z0-9_]{4,20}@163|126|qq.com", "hello@qq.com")  # 不能匹配出来子串

# 2-3 (ab) 将括号中字符作为一个分组
# result = re.match("[a-zA-Z0-9_]{4,20}@(163|126|qq).com", "hello@qq.com")
# result = re.match("[a-zA-Z0-9_]{4,20}@(163|126|qq).com", "hello@qqxcom")
result = re.match("[a-zA-Z0-9_]{4,20}@(163|126|qq)\.com", "hello@qq.com")  # 需要使用转义字符

info = result.group()

print('result-->', result)
print(info)
```

```
程序运行结果:
    result--> <re.Match object; span=(0, 12), match='hello@qq.com '>
    hello@qq.com
```

黑马程序员
www.itheima.com

2、举个栗子 -3

```
# 3 需求：匹配qq:10567这样的数据，提取出来qq文字和qq号码
# group(0)/group() 代表的是匹配的所有数据 1:第一个分组的数据 2:第二个分组的数据 顺序是从左到右依次排序的
# result = re.match("(qq):([0-9]\d{4,11})", "qq:10567") # 一般qq号不以0开头
result = re.match("(qq):([1-9]\d{4,11})", "qq:10567")
if result:
    info = result.group(0)
    print(info)

    num = result.group(2)
    print(num)

    type = result.group(1)
    print(type)
else:
    print("匹配失败")
```

```
程序运行效果
    qq:10567
    10567
    qq
```

黑马程序员
www.itheima.com

2、举个栗子 -3

```
# 3 需求：匹配qq:10567这样的数据，提取出来qq文字和qq号码
# group(0)/group() 代表的是匹配的所有数据 1:第一个分组的数据 2:第二个分组的数据 顺序是从左到右依次排序的
# result = re.match("(qq):([0-9]\d{4,11})", "qq:10567") # 一般qq号不以0开头
result = re.match("(qq):([1-9]\d{4,11})", "qq:10567")
if result:
    info = result.group(0)
    print(info)

    num = result.group(2)
    print(num)

    type = result.group(1)
    print(type)
else:
    print("匹配失败")
```

```
程序运行效果
    qq:10567
    10567
    qq
```

正则表达式编写

2、举个栗子 -4

```
# 有关分组, 分组的引用, 给分组起个别名
# 4 需求：匹配出<html>hh</html>
# \num 引用分组num匹配到的字符串
# result = re.match("<([a-zA-Z1-6]{4})>.*</([a-zA-Z1-6]{4})>", "<html>hh</html>")
# result = re.match("<([a-zA-Z1-6]{4})>.*</\\1>", "<html>hh</html>")
result = re.match(r"<([a-zA-Z1-6]{4})>.*</\1>", "<html>hh</html>")  # 前面加1个r,也不用转义了
if result:
    info = result.group()
    print(info)
else:
    print("匹配失败")

# 测试打印,比较不同
# print('\1')
# print('\\1')
```

程序运行效果　<html>hh</html>

黑马程序员
www.itheima.com

2、总结

总结

代码	功能
|	匹配左右任意一个表达式
(ab)	将括号中字符作为一个分组
\num	引用分组num匹配到的字符串

练习

①需求：在列表中["apple", "banana", "orange", "pear"]，匹配apple和pear

②需求：匹配出163、126、qq等邮箱

③需求：匹配qq:10567这样的数据，提取出来qq文字和qq号码

④需求：匹配出<html>hh</html>

⑤需求：匹配出<html><h1>www.itcast.cn</h1></html>

黑马程序员
www.itheima.com
传智教育旗下高端IT教育品牌